11-768 HW2: Evaluating a Data-Visualization Agent

Shengzhong Guan

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

The baseline performed poorly overall (macro-MCC 0.0268). Table 1 shows the family scores. Its execution judge produced 33 false positives and eight false negatives. It predicted no `wrong_data` or `hard_to_read` errors, and found only three of 71 evaluated `wrong_chart` positives.

Table 1: Seed-set MCC from `output/baseline_predictions.json`, `pre_iteration_seed_predictions.json`, and `final_seed_predictions.json`.

Validator	Execution	Wrong data	Wrong chart	Hard to read	Macro
Baseline	-0.0022	0.0000	0.1096	0.0000	0.0268
Part 2	1.0000	0.4251	0.3088	0.0301	0.4410
Final	1.0000	0.3906	0.2975	0.0301	0.4296

1.2 Recurring Error Type 1: Contradictory Execution Decisions

The baseline asked the VLM whether execution failed and parsed the first `YES`/`NO` token. In runs 004, 022, and 023, its own evidence said that the agent completed and produced `figure.png`, yet it emitted `execution_failure`. Because this family caused an early return, the false positive also suppressed the other three judges; run 004, for example, was human-labeled with both wrong data and wrong chart. Conversely, all twelve true execution failures (001, 010, 053, 065, 089, 096, 103, 145, 168, 171, 181, and 208) lacked the required root `figure.png`, but the baseline missed eight of them.

1.3 Recurring Error Type 2: Broad Prompts Miss Visual Defects

The baseline's short independent questions did not force a requirement-by- requirement comparison. Run 008 contained nearly white node labels on a white background and omitted a required orange source label. Run 009 left dense date ticks horizontal rather than at the requested 30-degree rotation, producing an overlapping band. Run 025 lacked the requested three-level, region-colored hierarchy and had extensive label collisions. Run 030 rendered one requested series white on white and replaced the exact suptitle. The baseline accepted all four.

1.4 Proposed Changes

First, replace model-based execution classification with deterministic checks for the required file, PNG format, and successful decoding. Second, make one joint structured VLM decision for the

remaining families using the complete rubric, explicit family boundaries, the task, input contents, trajectory, and rendered image. Require concrete evidence for every positive decision.

2 Part 2: Improve the Validator

2.1 Implemented Changes

The implementation follows both proposals. `judge_execution` uses Pillow to reject a missing, corrupt, or non-PNG output and otherwise returns an empty list, allowing later checks to run. The nonterminal judge sends a bounded excerpt of input and trajectory evidence plus a downscaled image and original pixel/DPI metadata in one request. Its prompt enumerates wrong values, selection, ordering, chart constraints, and concrete readability failures. It expects a JSON object containing one Boolean and evidence string per family, and validates that schema before constructing errors. This also avoids the baseline's brittle interpretation of a leading word that contradicts the explanation.

2.2 Seed-Set Performance and Qualitative Analysis

The full Part 2 results appear in Table 1. Execution reached TP=12, TN=98, FP=0, and FN=0, eliminating both contradictory decisions and the terminal cascade. Wrong-data MCC rose from 0 to 0.4251 and wrong-chart MCC from 0.1096 to 0.3088. The overall macro-MCC rose from 0.0268 to 0.4410. However, readability remained the main weakness: the judge found only one of 39 positives (MCC 0.0301).

The revised rubric correctly recovered multiple defects in runs 004, 025, and 030. A representative false negative is run 008: the image visibly contains near-white labels on white, but the model returned only `wrong_chart` and missed `hard_to_read`. A representative false positive is run 013, where it called requested axis-limit/design discrepancies `wrong_data`; this suggests that the family boundary needs more emphasis. Run 031 is another false positive: the model's own explanation first claims the legend mapping is wrong and then says the same mapping is correct, showing that structured output alone does not eliminate visual hallucination.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

I targeted two classes. *Derived-data encoding* tests whether the validator can verify calculations that are not directly readable from the prompt image alone, especially when several visual channels encode different derived quantities. *Precise chart constraints* tests whether it can audit exact scales, styles, subplot relationships, annotations, and layout instead of merely recognizing the broad chart type.

3.2 Task Design

- `regional_profit_filter`: filter records, compute and aggregate profit, sort descending, and draw labeled horizontal bars.
- `weighted_bubble_metrics`: encode latency, accuracy, request count, and cost simultaneously through position, area, and color.

- `monthly_conversion_combo`: derive conversion rates and a cumulative series in stacked panels, including a twin axis and annotations.
- `log_errorbar_threshold`: combine log scales, proportional error bars, exact limits, shading, grids, and an arrow annotation.
- `shared_panel_functions`: enforce four exact functions in a shared-axis 2-by-2 layout with panel-specific styles and spine rules.

Every task's inputs are empty: all data are synthetic, written by me, and included directly in the prompts, making the expected values independently checkable without external data or packages. Each task was run once with Qwen2.5-Coder-3B, Ministral-3-14B, and GLM-4.7-Flash through the fixed scaffold.

3.3 Agent Runs, Human Labels, and Validator Performance

I reviewed each task instruction, complete trajectory, and final figure, assigned an empty list only when it met the rubric, and treated a missing/invalid figure as terminal. I also read each PNG's pixel dimensions. This caught an easy-to-miss failure: several agents created the requested Matplotlib canvas but saved with `bbox_inches="tight"`, changing the final required pixel dimensions. Those runs are labeled `wrong_chart`, consistent with the rubric's exact figure-size requirement.

The Part 2 validator retained perfect execution classification but had wrong-data MCC -0.2000 because it missed both horizontal-bar ordering errors and hallucinated two other data errors. It detected eight of eleven chart errors without a false positive (MCC 0.4264). Readability MCC was -0.0909: it missed the real overlapping twin-axis labels in `monthly_conversion_combo__qwen` and hallucinated clipping in a readable grid.

The agent failures and validator failures were distinct. Qwen's three missing figures were agent execution failures; its incomplete monthly annotations and the two non-Qwen `barh` reversals were also genuine agent mistakes. Tight-bounding-box saves produced genuine size violations across otherwise strong figures. By contrast, claims that correct labels were missing or that a correct cubic series used wrong values were validator hallucinations. The former require better generation behavior; the latter motivate conservative, grounded validator checks.

Table 2: Authored-set MCC.

Validator	Execution	Wrong data	Wrong chart	Hard to read	Macro
Part 2	1.0000	-0.2000	0.4264	-0.0909	0.2839
Final	1.0000	0.6325	1.0000	1.0000	0.9081

Ambiguous labels. Two judgments were close calls. A figure saved with `bbox_inches="tight"` meets the Matplotlib canvas request but not the final pixel size; I labeled it `wrong_chart` because the task states the saved dimensions. Overlapping twin-axis tick labels in `monthly_conversion_combo__qwen` were labeled `hard_to_read` because the text is present but difficult to read.

3.4 Harbor Packaging

Each task asks for `plot.py`, `figure.png`, and a task-specific `plotted_values.json`. Its deterministic tests check file presence, PNG dimensions, literal expected sidecar values, and reproducibility

after rerunning `plot.py`. All five reference solutions scored 1.0. The `wrong_total` mutant changes West's computed profit from 90 to 88; it scored 0 because `test_plotted_values_are_exact` rejected the changed value. The `illegible` mutant preserves the values and dimensions but uses 2-point text; it scored 1 because the verifier does not measure rendered legibility.

For these tasks, execution failure and many exact data values can be decided deterministically. Some requested chart properties can also be checked when represented in code or metadata, but arbitrary visual design errors and `hard_to_read` failures generally require image interpretation. Moreover, a per-task verifier contains hand-derived answers and therefore cannot grade private runs from unseen tasks; the submitted validator must infer requirements from each new prompt and its evidence.

4 Part 4: Iterate on the Design

4.1 Additional Improvement Motivated by Part 3

Part 3 exposed two requirements that were more reliable to verify from metadata and code than from pixels. First, the joint VLM missed exact saved-image sizes. I added a conservative parser for explicit "width by height inches at DPI" requirements and compare their computed pixel dimensions with the decoded PNG. Second, both non-Qwen agents sorted values descending before calling `barh` but did not reverse the list or call `invert_yaxis`; Matplotlib therefore displayed the largest value at the bottom. I added a trajectory check only when the prompt explicitly requires a descending horizontal chart from top to bottom and the uncorrected code pattern is present. Both checks emit grounded evidence and merge with, rather than replace, the model judgment.

4.2 Final Evaluation and Qualitative Error Analysis

On the seed set, the final validator scored macro-MCC 0.4296 (Table 1), slightly below Part 2 (0.4410): the two new checks key on phrases specific to my authored prompts and rarely fire there, so the difference reflects sampling variation in the VLM's answers (17 of 110 runs changed prediction between the two sweeps, with both gains and losses, e.g. 191 gained a correct `hard_to_read` while 004 and 050 lost a correct `wrong_data`). The seed set therefore shows no regression attributable to the deterministic checks, but also no improvement; the gains below are on the authored set.

On the authored set, macro-MCC increased from 0.2839 to 0.9081. Wrong-data MCC rose from -0.2000 to 0.6325 because the deterministic `barh` check found both previously missed ordering errors. Wrong-chart MCC reached 1.0 because the dimension check found all three previous false negatives without adding a false positive. In this final run, the model also correctly classified the one readability-positive run, so that family reached MCC 1.0; because its evidence focused on a clipped annotation rather than the most salient overlapping twin ticks, I do not treat this small-set result as proof of robust readability.

The final authored set has two wrong-data false positives and no false negatives. For `shared_panel_functions__`, the judge claimed the cubic points do not match $[-8, -1, 0, 1, 8]$, although those exact values are visible and generated by the trajectory. For `monthly_conversion_combo__qwen`, it claimed the computed conversion percentages were wrong even though the executed formula was correct. Both indicate residual VLM visual hallucination; the deterministic checks improve requirements with safe programmatic interpretations but do not attempt to replace arbitrary numerical reasoning.

4.3 Further Changes and Reproduction

I made no validator changes beyond the two deterministic checks above. Because the model's sampling is not deterministic, results can vary between sweeps, as the authored-set rerun shows. Experiments were run with `python -m validator.runner -artifacts artifacts|runs -out output/...` followed by `python -m workflow score`; the outputs are in `output/`.